Eduvos Assessment Coversheet

Academic Details

Campus: **Online**

Faculty: **Information Technology**

Module Code: **ITNPA4-34**

Lecturer's Name: **Tinotenda Mavetera**

Group (if applicable) **1**

Assessment Details

Assessment Type: **Individual Assessment**

Assignment/Brief Number:

Due Date/ Submission Date: **13 November 2026**

Student Details (Group, if applicable)

#	Full Name	Student Number	Contact Number	% Participation	Signature
1					
2					
3					
4					
5					
6					

Student Details (Individual, if applicable)

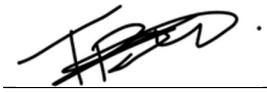
Student Full Name: **Thabo Baleni**

Student Number: **EDUV4818947**

Academic Integrity

AI Disclosure Completed	[X] **Yes**	[ ] **No**

Declaration:
I declare that this assessment is my own original work except for source material explicitly acknowledged. I confirm that this work has not previously been submitted for assessments in any course. I acknowledge the institutional AI policy and confirm that AI tools have only been used as stipulated and documented in the AI Disclosure Appendix.

Student Signature 

Date **01 November 2026**

Lecturer Use Only

Lecturer Comments:

Marks Awarded (%):

Lecturer Signature:

Date:

Table of Contents

Eduvos Assessment Coversheet 1
Assessment 4
Question 1 4
1.1) 4
1.2) 10
1.3) 15
Question 2 25
2.1) 25
2.2) 31
2.3) 32
Question 3 36
3.1) 44
3.2) 45
3.3) 47
3.4) 51
Question 4 57
4.1) 57
4.2) 60
AI Evidence Section 62
AI Disclosure Appendix 63
Declaration: 63

Assessment

Question 1

1.1)

```
import pandas as pd
import json
import warnings

warnings.filterwarnings('ignore')

with open('research_abstracts.json', 'r', encoding='utf-8') as file:
    data = json.load(file)

# JSON data into a DataFrame
df = pd.DataFrame(data)

df.head()
```

Output

[1]:

	DOI	Abstract	Label
0	10.1155/2021/1071145	Forward-looking forecasting of the inflation r...	ai
1	10.1007/s43546-022-00384-2	Our study examined the disaggregation of infla...	ai
2	10.1002/for.2667	This paper develops a dynamic factor model tha...	classical
3	10.1007/s42973-021-00091-x	In this paper, I use a simple SIR Macro model ...	classical
4	10.1007/s40258-012-0003-z	BackgroundGame theory is useful for identifyin...	classical

```
# Dataset dimensions
print("Dataset shape:", df.shape)

# Column names and data types
```

```
print("\nDataset information:")
df.info()

# Missing values
print("\nMissing values:")
print(df.isnull().sum())

# Duplicate records
print("\nDuplicate records:", df.duplicated().sum())

# Distribution of research categories
print("\nLabel distribution:")
print(df['Label'].value_counts())
```

Output:

```
Dataset shape: (1138, 3)

Dataset information:
<class 'pandas.core.frame.DataFrame'>
Index: 1138 entries, 0 to 1137
Data columns (total 3 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   DOI       1134 non-null   object
 1   Abstract  1138 non-null   object
 2   Label     1138 non-null   object
dtypes: object(3)
memory usage: 35.6+ KB

Missing values:
DOI         4
Abstract    0
Label       0
dtype: int64

Duplicate records: 2

Label distribution:
Label
classical    900
ai           238
Name: count, dtype: int64
```

```
# Removing duplicate records
df = df.drop_duplicates()

# Resetting the DataFrame index
df = df.reset_index(drop=True)

print("Dataset shape after removing duplicates:", df.shape)
print("Remaining duplicate records:", df.duplicated().sum())
```

Output:

```
Dataset shape after removing duplicates: (1136, 3)
Remaining duplicate records: 0
```

```
import re
```

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

# Text denoising techniques
# NLTK resources
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')



stop_words = set(stopwords.words('english'))



stop_words = stop_words - {'no', 'not', 'nor'}

# Text preprocessing function
def preprocess(text):


    clean_text = text.lower()



    clean_text = re.sub(r'[^a-zA-Z\s]', ' ', clean_text)



    clean_text = re.sub(r'\s+', ' ', clean_text).strip()
```

```
    words = word_tokenize(clean_text)


    filtered_words = [
        word for word in words
        if word not in stop_words
    ]


    clean_text = ' '.join(filtered_words)

    return clean_text


# Preprocessing to the Abstract column
df['Clean_Abstract'] = df['Abstract'].apply(preprocess)

# Original and cleaned abstracts
df[['Abstract', 'Clean_Abstract', 'Label']].head()
```

Output:

[4]:

	Abstract	Clean_Abstract	Label
0	Forward-looking forecasting of the inflation r...	forward looking forecasting inflation rate cou...	ai
1	Our study examined the disaggregation of infla...	study examined disaggregation inflation compon...	ai
2	This paper develops a dynamic factor model tha...	paper develops dynamic factor model uses euro ...	classical
3	In this paper, I use a simple SIR Macro model ...	paper use simple sir macro model examine japan...	classical
4	BackgroundGame theory is useful for identifyin...	backgroundgame theory useful identifying condi...	classical

```
print("Original abstract:")
print(df['Abstract'].iloc[0])
```

```
print("\nCleaned abstract:")
print(df['Clean_Abstract'].iloc[0])

# The length of the original and cleaned abstracts
df['Original_Length'] = df['Abstract'].str.len()
df['Cleaned_Length'] = df['Clean_Abstract'].str.len()

print("\nAverage original abstract length:",
      df['Original_Length'].mean())

print("Average cleaned abstract length:",
      df['Cleaned_Length'].mean())


print("\nFinal dataset shape:", df.shape)
df.head()
```

Output:

```
Original abstract:
Forward-looking forecasting of the inflation rate could help the central bank and other government departments to better use monetary policy to stabili
ze prices and prevent the impact of inflation on market entities, especially for low- and middle-income groups. It can also help financial institutions
and investors better make investment decisions. In this sense, the forecast of inflation rate is of great significance. The existing literature mainly
uses linear models such as autoregressive (AR) and vector autoregressive (VAR) models to predict the inflation rate. The nonlinear relationship between
variables and the mining of historical data information are relatively lacking. Therefore, the prediction strategies and accuracy of the existing liter
ature need to be improved. The predictive model designed in deep learning can fully mine the nonlinear relationship between variables and process compl
ex long-term time series dynamic information, thereby making up for the deficiencies of existing research. Therefore, this paper employs the recurrent
neural networks with gated recurrent unit (GRU-RNN) model to train and analyze the Consumer Price Index (CPI) indicators to obtain inflation-related pr
ediction results. The experimental results on historical data show that the GRU-RNN model has good performance in predicting China's inflation rate. In
comparison, the performance of the proposed method is significantly better than some traditional models, showing its superior effectiveness.

Cleaned abstract:
forward looking forecasting inflation rate could help central bank government departments better use monetary policy stabilize prices prevent impact in
flation market entities especially low middle income groups also help financial institutions investors better make investment decisions sense forecast
inflation rate great significance existing literature mainly uses linear models autoregressive ar vector autoregressive var models predict inflation ra
te nonlinear relationship variables mining historical data information relatively lacking therefore prediction strategies accuracy existing literature
need improved predictive model designed deep learning fully mine nonlinear relationship variables process complex long term time series dynamic informa
tion thereby making deficiencies existing research therefore paper employs recurrent neural networks gated recurrent unit gru rnn model train analyze c
onsumer price index cpi indicators obtain inflation related prediction results experimental results historical data show gru rnn model good performance
predicting china inflation rate comparison performance proposed method significantly better traditional models showing superior effectiveness

Average original abstract length: 1145.0765845070423
Average cleaned abstract length: 860.8089788732394

Final dataset shape: (1136, 6)
```

5]:

	DOI	Abstract	Label	Clean_Abstract	Original_Length	Cleaned_Length
0	10.1155/2021/1071145	Forward-looking forecasting of the inflation r...	ai	forward looking forecasting inflation rate cou...	1502	1199
1	10.1007/s43546-022-00384-2	Our study examined the disaggregation of infla...	ai	study examined disaggregation inflation compon...	1218	919
2	10.1002/for.2667	This paper develops a dynamic factor model tha...	classical	paper develops dynamic factor model uses euro ...	1083	851
3	10.1007/s42973-021-00091-x	In this paper, I use a simple SIR Macro model ...	classical	paper use simple sir macro model examine japan...	1251	903
4	10.1007/s40258-012-0003-z	BackgroundGame theory is useful for identifyin...	classical	backgroundgame theory useful identifying condi...	1576	1253

1.2)

```
# Input feature
texts = df['Clean_Abstract'].astype(str).values


# Target variable
labels = df['Label'].values


# Selected features
print("Number of abstracts:", len(texts))
print("Number of labels:", len(labels))


print("\nInput feature example:")
print(texts[0][:500])


print("\nTarget label example:")
print(labels[0])
```

Output:

```
Number of abstracts: 1136
Number of labels: 1136

Input feature example:
forward looking forecasting inflation rate could help central bank government departments better use monetary policy stabilize prices prevent impact in
flation market entities especially low middle income groups also help financial institutions investors better make investment decisions sense forecast
inflation rate great significance existing literature mainly uses linear models autoregressive ar vector autoregressive var models predict inflation ra
te nonlinear relationship variables mining hist

Target label example:
ai
```

```
import numpy as np

# Categorical labels into numerical values
label_mapping = {
    'classical': 0,
    'ai': 1
}

y = np.array([label_mapping[label] for label in labels])

print("Label mapping:", label_mapping)

print("\nEncoded labels:")
print(y[:10])

print("\nClass distribution:")
print(pd.Series(y).value_counts())
```

Output:

```
Label mapping: {'classical': 0, 'ai': 1}

Encoded labels:
[1 1 0 0 0 1 0 0 0 1]

Class distribution:
0    900
1    236
Name: count, dtype: int64
```

```
from sklearn.model_selection import train_test_split

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    texts,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

print("\nTraining class distribution:")
print(pd.Series(y_train).value_counts())

print("\nTesting class distribution:")
print(pd.Series(y_test).value_counts())
```

Output:

```
Training samples: 908
Testing samples: 228

Training class distribution:
0    719
1    189
Name: count, dtype: int64

Testing class distribution:
0    181
1     47
Name: count, dtype: int64
```

```
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Feature parameters
max_features = 10000
maxlen = 500

# Tokenizer
tokenizer = Tokenizer(
    num_words=max_features,
    oov_token="<OOV>"
)



tokenizer.fit_on_texts(X_train)

# Converting text into numerical sequences
X_train_sequences = tokenizer.texts_to_sequences(X_train)
X_test_sequences = tokenizer.texts_to_sequences(X_test)

# Padding
```

```
X_train_padded = pad_sequences(
    X_train_sequences,
    maxlen=maxlen,
    padding='post',
    truncating='post'
)

X_test_padded = pad_sequences(
    X_test_sequences,
    maxlen=maxlen,
    padding='post',
    truncating='post'
)



print("Training feature shape:", X_train_padded.shape)
print("Testing feature shape:", X_test_padded.shape)

print("\nNumber of unique words:", len(tokenizer.word_index))
```

Output:

```
Training feature shape: (908, 500)
Testing feature shape: (228, 500)

Number of unique words: 8557
```

```
# Numerical sequences
print("Original cleaned abstract:")
print(X_train[0][:300])

print("\nNumerical sequence:")
```

```
print(X_train_sequences[0][:30])

print("\nPadded sequence:")
print(X_train_padded[0][:30])

print("\nTraining target:")
print(y_train[0])
```

Output:

```
Original cleaned abstract:
fact predictions inflation create self fulfilling expectations trap creates powerful incentive balance output inflation media reports source public inf
ormation contribute formation inflation expectations investigate media coverage possible media bias may affect inflation expectations using sticky in

Numerical sequence:
[479, 9, 2, 2089, 2090, 3478, 38, 1345, 1773, 2091, 1774, 596, 37, 2, 1205, 1436, 1067, 380, 49, 1437, 779, 2, 38, 331, 1205, 1132, 399, 1205, 338, 87]

Padded sequence:
[ 479    9    2 2089 2090 3478   38 1345 1773 2091 1774  596   37    2
 1205 1436 1067  380   49 1437  779    2   38  331 1205 1132  399 1205
  338   87]

Training target:
1
```

1.3)

```
import numpy as np
import matplotlib.pyplot as plt

import tensorflow as tf

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense

from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay
)
```

```
# Random seeds for reproducibility
np.random.seed(42)
tf.random.set_seed(42)


model = Sequential()


model.add(tf.keras.Input(shape=(maxlen,)))

# Embedding layer
model.add(Embedding(
    input_dim=max_features,
    output_dim=32
))

# LSTM layer
model.add(LSTM(
    128,
    dropout=0.2,
    recurrent_dropout=0.2
))

# Output layer
model.add(Dense(
    1,
    activation='sigmoid'
))
```

```
# Compile model
model.compile(
    optimizer='rmsprop',
    loss='binary_crossentropy',
    metrics=['accuracy']
)


model.summary()
```

Output:

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 500, 32)	320,000
lstm (LSTM)	(None, 128)	82,432
dense (Dense)	(None, 1)	129

```
Total params: 402,561 (1.54 MB)

Trainable params: 402,561 (1.54 MB)

Non-trainable params: 0 (0.00 B)
```

```
# Training the LSTM model
history = model.fit(
    X_train_padded,
    y_train,
    epochs=10,
    batch_size=60,
    validation_split=0.2,
```

```
    verbose=1
)
```

Output:

```
Epoch 1/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 11s 427ms/step - accuracy: 0.7920 - loss: 0.5531 - val_accuracy: 0.7857 - val_loss: 0.5246
Epoch 2/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 5s 377ms/step - accuracy: 0.7934 - loss: 0.5146 - val_accuracy: 0.7857 - val_loss: 0.5339
Epoch 3/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 5s 393ms/step - accuracy: 0.7934 - loss: 0.5159 - val_accuracy: 0.7857 - val_loss: 0.5216
Epoch 4/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 5s 378ms/step - accuracy: 0.7934 - loss: 0.5172 - val_accuracy: 0.7857 - val_loss: 0.5348
Epoch 5/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 5s 389ms/step - accuracy: 0.7934 - loss: 0.5150 - val_accuracy: 0.7857 - val_loss: 0.5299
Epoch 6/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 5s 387ms/step - accuracy: 0.7934 - loss: 0.5116 - val_accuracy: 0.7857 - val_loss: 0.5243
Epoch 7/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 5s 392ms/step - accuracy: 0.7934 - loss: 0.5129 - val_accuracy: 0.7857 - val_loss: 0.5221
Epoch 8/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 5s 397ms/step - accuracy: 0.7934 - loss: 0.5118 - val_accuracy: 0.7857 - val_loss: 0.5237
Epoch 9/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 5s 399ms/step - accuracy: 0.7934 - loss: 0.5121 - val_accuracy: 0.7857 - val_loss: 0.5241
Epoch 10/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 5s 390ms/step - accuracy: 0.7934 - loss: 0.5118 - val_accuracy: 0.7857 - val_loss: 0.5246
```

```
# Performance history
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

loss = history.history['loss']
val_loss = history.history['val_loss']

epochs = range(1, len(acc) + 1)

# Training and validation accuracy
plt.figure(figsize=(10, 5))

plt.plot(epochs, acc, label='Training Accuracy')
plt.plot(epochs, val_acc, label='Validation Accuracy')

plt.title('Training and Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
```

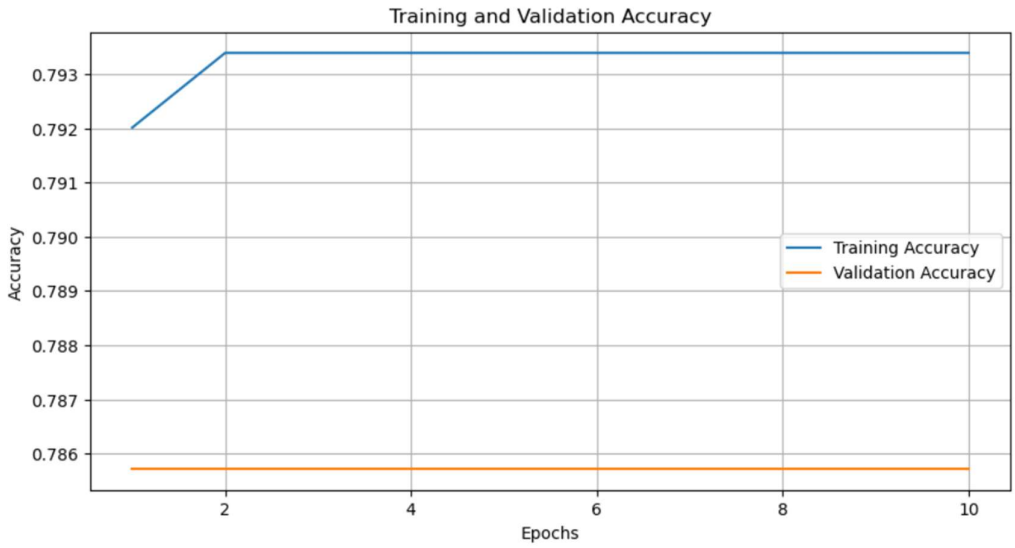
```
plt.legend()
plt.grid(True)
plt.show()

# Training and validation loss
plt.figure(figsize=(10, 5))

plt.plot(epochs, loss, label='Training Loss')
plt.plot(epochs, val_loss, label='Validation Loss')

plt.title('Training and Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)
plt.show()
```

Output:






```
# The model on the test dataset
test_loss, test_accuracy = model.evaluate(
    X_test_padded,
    y_test,
    verbose=1
)

print("Test Loss:", round(test_loss, 4))
print("Test Accuracy:", round(test_accuracy, 4))
print("Test Accuracy (%):", round(test_accuracy * 100, 2), "%")
```

Output:

```
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 55ms/step - accuracy: 0.7939 - loss: 0.5098
Test Loss: 0.5098
Test Accuracy: 0.7939
Test Accuracy (%): 79.39 %
```

```
# Prediction probabilities
y_pred_probability = model.predict(X_test_padded)

# Probabilities into binary predictions
```

```
y_pred = (y_pred_probability >= 0.5).astype(int).ravel()

# Accuracy
print("Accuracy:", round(accuracy_score(y_test, y_pred), 4))

# Classification report
print("\nClassification Report:\n")

print(classification_report(
    y_test,
    y_pred,
    target_names=['Classical', 'AI'],
    zero_division=0
))
```

Output:

```
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 103ms/step
Accuracy: 0.7939

Classification Report:

              precision    recall  f1-score   support

   Classical       0.79      1.00      0.89       181
          AI       0.00      0.00      0.00        47

    accuracy                           0.79       228
   macro avg       0.40      0.50      0.44       228
weighted avg       0.63      0.79      0.70       228
```

```
# Confusion matrix
cm = confusion_matrix(y_test, y_pred)

# Confusion matrix
```

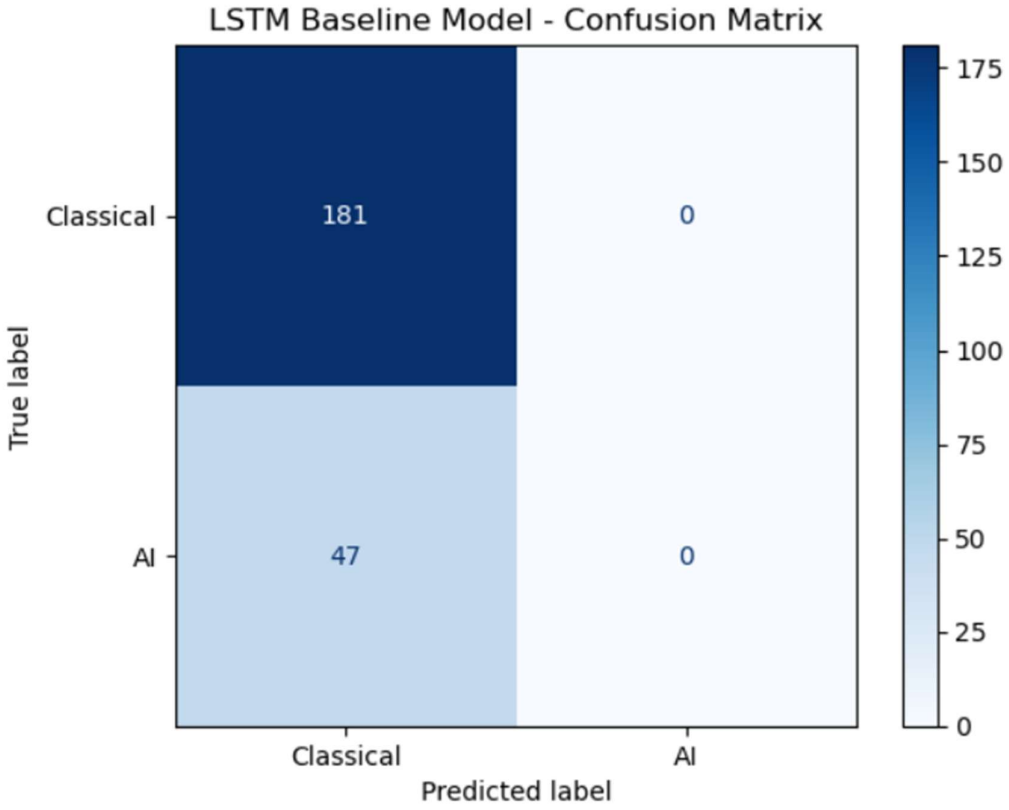
```
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=['Classical', 'AI']
)

disp.plot(cmap='Blues', values_format='d')

plt.title('LSTM Baseline Model - Confusion Matrix')
plt.show()

print("Confusion Matrix:")
print(cm)
```

Output:




```
Confusion Matrix:
[[181   0]
 [ 47   0]]
```

```
# Final training and validation metrics
final_train_accuracy = history.history['accuracy'][-1]
final_val_accuracy = history.history['val_accuracy'][-1]

final_train_loss = history.history['loss'][-1]
final_val_loss = history.history['val_loss'][-1]

# Performance summary
performance = pd.DataFrame({
    'Metric': [
        'Training Accuracy',
        'Validation Accuracy',
        'Test Accuracy',
```

```
        'Training Loss',
        'Validation Loss',
        'Test Loss'
    ],
    'Value': [
        final_train_accuracy,
        final_val_accuracy,
        test_accuracy,
        final_train_loss,
        final_val_loss,
        test_loss
    ]
})

display(performance)
```

Output:

	Metric	Value
0	Training Accuracy	0.793388
1	Validation Accuracy	0.785714
2	Test Accuracy	0.793860
3	Training Loss	0.511831
4	Validation Loss	0.524606
5	Test Loss	0.509801

Question 2

2.1)

#2.1

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.pipeline import Pipeline
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix
)

import pandas as pd
import numpy as np
import joblib
from pathlib import Path

# Four machine-learning models
models = {
    "Multinomial Naive Bayes": MultinomialNB(),
```

```
    "Logistic Regression": LogisticRegression(
        max_iter=1000,
        random_state=42
    ),

    "Support Vector Machine": LinearSVC(
        random_state=42
    ),

    "Random Forest": RandomForestClassifier(
        n_estimators=200,
        random_state=42,
        n_jobs=-1
    )
}

results = []
trained_models = {}

# Model evaluation
for name, classifier in models.items():

    pipeline = Pipeline([
        ("tfidf", TfidfVectorizer(
            max_features=20000,
            ngram_range=(1, 2),
            sublinear_tf=True
        )),
```

```
    ("classifier", classifier)
])

# Training model
pipeline.fit(X_train, y_train)

# Predict test data
predictions = pipeline.predict(X_test)

trained_models[name] = pipeline

# Performance metrics
results.append({
    "Model": name,
    "Accuracy": accuracy_score(y_test, predictions),
    "AI Precision": precision_score(
        y_test, predictions, pos_label=1, zero_division=0
    ),
    "AI Recall": recall_score(
        y_test, predictions, pos_label=1, zero_division=0
    ),
    "AI F1-Score": f1_score(
        y_test, predictions, pos_label=1, zero_division=0
    )
})

print("\n", "=" * 60)
print(name)
print("=" * 60)
```

```
    print(classification_report(
        y_test,
        predictions,
        labels=[0, 1],
        target_names=["Classical", "AI"],
        zero_division=0
    ))

    print("Confusion Matrix:")
    print(confusion_matrix(y_test, predictions, labels=[0, 1]))

# Model comparison
results_df = pd.DataFrame(results)

results_df = results_df.sort_values(
    by="AI F1-Score",
    ascending=False
).reset_index(drop=True)

print("\nMODEL PERFORMANCE COMPARISON")
display(results_df)
```

Output:

```
 ============================================================
Multinomial Naive Bayes
============================================================
              precision    recall  f1-score   support

   Classical       0.80      1.00      0.89       181
          AI       1.00      0.02      0.04        47

    accuracy                           0.80       228
   macro avg       0.90      0.51      0.46       228
weighted avg       0.84      0.80      0.71       228

Confusion Matrix:
[[181   0]
 [ 46   1]]

 ============================================================
Logistic Regression
============================================================
              precision    recall  f1-score   support

   Classical       0.84      1.00      0.91       181
          AI       1.00      0.26      0.41        47

    accuracy                           0.85       228
   macro avg       0.92      0.63      0.66       228
weighted avg       0.87      0.85      0.81       228

Confusion Matrix:
[[181   0]
 [ 35  12]]
```

```
Support Vector Machine
============================================================
              precision    recall  f1-score   support

   Classical       0.93      0.98      0.95       181
          AI       0.92      0.70      0.80        47

    accuracy                           0.93       228
   macro avg       0.92      0.84      0.87       228
weighted avg       0.92      0.93      0.92       228

Confusion Matrix:
[[178   3]
 [ 14  33]]

 ============================================================
Random Forest
============================================================
              precision    recall  f1-score   support

   Classical       0.89      1.00      0.94       181
          AI       1.00      0.51      0.68        47

    accuracy                           0.90       228
   macro avg       0.94      0.76      0.81       228
weighted avg       0.91      0.90      0.89       228

Confusion Matrix:
[[181   0]
 [ 23  24]]
```

MODEL PERFORMANCE COMPARISON

	Model	Accuracy	AI Precision	AI Recall	AI F1-Score
0	Support Vector Machine	0.925439	0.916667	0.702128	0.795181
1	Random Forest	0.899123	1.000000	0.510638	0.676056
2	Logistic Regression	0.846491	1.000000	0.255319	0.406780
3	Multinomial Naive Bayes	0.798246	1.000000	0.021277	0.041667

```
# Baseline performance from Question 1
baseline_results = {
    "Model": "Baseline LSTM (Question 1)",
    "Accuracy": accuracy_score(y_test, y_pred),
    "AI Precision": precision_score(
        y_test, y_pred, pos_label=1, zero_division=0
```

```
    ),
    "AI Recall": recall_score(
        y_test, y_pred, pos_label=1, zero_division=0
    ),
    "AI F1-Score": f1_score(
        y_test, y_pred, pos_label=1, zero_division=0
    )
}



baseline_comparison = pd.concat(
    [pd.DataFrame([baseline_results]), results_df],
    ignore_index=True
)

print("BASELINE AND MACHINE LEARNING MODEL COMPARISON")
display(baseline_comparison)
```

Output:

```
BASELINE AND MACHINE LEARNING MODEL COMPARISON
```

	Model	Accuracy	AI Precision	AI Recall	AI F1-Score
0	Baseline LSTM (Question 1)	0.793860	0.000000	0.000000	0.000000
1	Support Vector Machine	0.925439	0.916667	0.702128	0.795181
2	Random Forest	0.899123	1.000000	0.510638	0.676056
3	Logistic Regression	0.846491	1.000000	0.255319	0.406780
4	Multinomial Naive Bayes	0.798246	1.000000	0.021277	0.041667

2.2)

#2.2

```
# Enhanced SVM using balanced class weights
balanced_svm = Pipeline([
  ("tfidf", TfidfVectorizer(
    max_features=20000,
    ngram_range=(1, 2),
    sublinear_tf=True
  )),

  ("classifier", LinearSVC(
    class_weight="balanced",
    random_state=42
  ))
])

# Training enhanced model
balanced_svm.fit(X_train, y_train)

# Predictions
balanced_predictions = balanced_svm.predict(X_test)

print("Enhanced SVM model trained successfully.")
```

Output:

```
Enhanced SVM model trained successfully.
```

```
# Original SVM predictions
original_svm = trained_models["Support Vector Machine"]
```

```
original_predictions = original_svm.predict(X_test)


# Performance
comparison_df = pd.DataFrame([
    {
        "Model": "Original SVM",
        "Accuracy": accuracy_score(y_test, original_predictions),
        "AI Precision": precision_score(
            y_test, original_predictions, pos_label=1, zero_division=0
        ),
        "AI Recall": recall_score(
            y_test, original_predictions, pos_label=1, zero_division=0
        ),
        "AI F1-Score": f1_score(
            y_test, original_predictions, pos_label=1, zero_division=0
        )
    },
    {
        "Model": "Enhanced SVM - Balanced Class Weights",
        "Accuracy": accuracy_score(y_test, balanced_predictions),
        "AI Precision": precision_score(
            y_test, balanced_predictions, pos_label=1, zero_division=0
        ),
        "AI Recall": recall_score(
            y_test, balanced_predictions, pos_label=1, zero_division=0
        ),
        "AI F1-Score": f1_score(
            y_test, balanced_predictions, pos_label=1, zero_division=0
```

```
        )
    }
])

print("PERFORMANCE ENHANCEMENT COMPARISON")
display(comparison_df)
```

Output:

PERFORMANCE ENHANCEMENT COMPARISON

	Model	Accuracy	AI Precision	AI Recall	AI F1-Score
0	Original SVM	0.925439	0.916667	0.702128	0.795181
1	Enhanced SVM - Balanced Class Weights	0.934211	0.900000	0.765957	0.827586

```
print("ENHANCED SVM CLASSIFICATION REPORT")

print(classification_report(
    y_test,
    balanced_predictions,
    labels=[0, 1],
    target_names=["Classical", "AI"],
    zero_division=0
))

print("Enhanced SVM Confusion Matrix:")

print(confusion_matrix(
    y_test,
    balanced_predictions,
    labels=[0, 1]
))
```

Output:

```
ENHANCED SVM CLASSIFICATION REPORT
              precision    recall  f1-score   support

   Classical       0.94      0.98      0.96       181
          AI       0.90      0.77      0.83        47

    accuracy                           0.93       228
   macro avg       0.92      0.87      0.89       228
weighted avg       0.93      0.93      0.93       228

Enhanced SVM Confusion Matrix:
[[177   4]
 [ 11  36]]
```

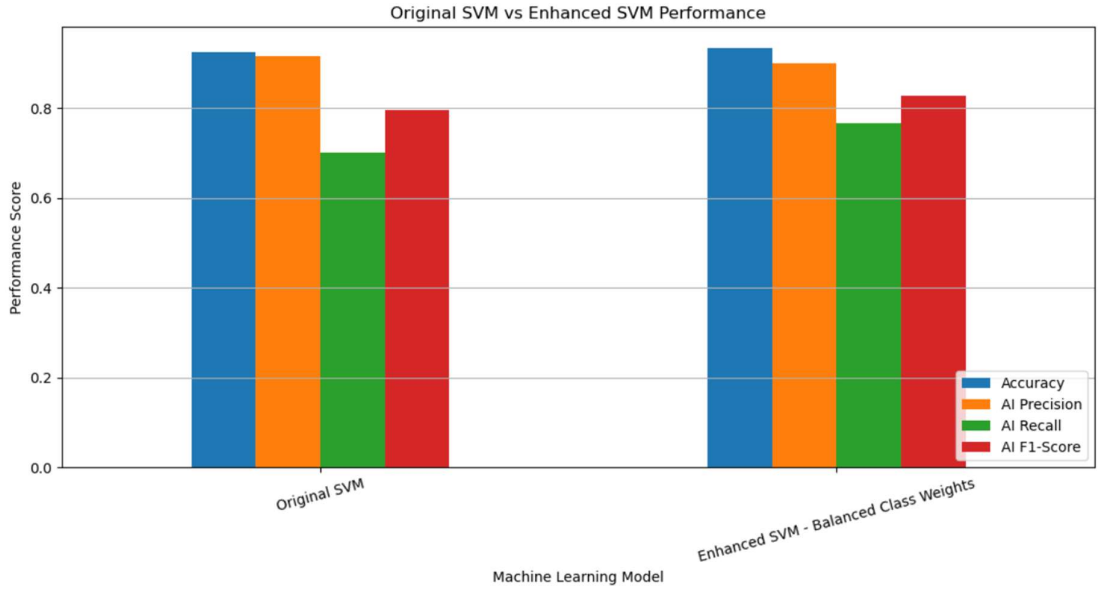
#Visualization of performance improvement

```
import matplotlib.pyplot as plt

comparison_df.plot(
    x="Model",
    y=["Accuracy", "AI Precision", "AI Recall", "AI F1-Score"],
    kind="bar",
    figsize=(11, 6)
)

plt.title("Original SVM vs Enhanced SVM Performance")
plt.xlabel("Machine Learning Model")
plt.ylabel("Performance Score")
plt.xticks(rotation=15)
plt.legend(loc="lower right")
plt.grid(axis="y")
plt.tight_layout()
plt.show()
```

Output:



2.3)

#2.3

All four models and the enhanced SVM

candidate_models = trained_models.copy()

candidate_models["Enhanced SVM"] = balanced_svm

Evaluation of each candidate

best_model_name = None

best_f1 = -1

for name, candidate in candidate_models.items():

  predictions = candidate.predict(X_test)

  score = f1_score(

    y_test,

    predictions,

```
        pos_label=1,
        zero_division=0
    )

    if score > best_f1:
        best_f1 = score
        best_model_name = name

best_model = candidate_models[best_model_name]

print("Best performing model:", best_model_name)
print("Best AI F1-score:", round(best_f1, 4))

# Deployment folder
deployment_folder = Path("deployment")
deployment_folder.mkdir(exist_ok=True)

# Save
model_path = deployment_folder / "best_research_classifier.pkl"

joblib.dump(best_model, model_path)

print("Model saved successfully:", model_path.resolve())
```

```
Best performing model: Enhanced SVM
Best AI F1-score: 0.8276
Model saved successfully: C:\Users\Tbose\deployment\best_research_classifier.pkl
```

#Streamlit

```
import os
```

```
os.makedirs("deployment", exist_ok=True)

app_code = r'''
import re
import joblib
import nltk
import streamlit as st

from pathlib import Path
from nltk.corpus import stopwords

nltk.download("stopwords", quiet=True)

STOP_WORDS = set(stopwords.words("english")) - {"no", "not", "nor"}

model_path = Path(__file__).parent / "best_research_classifier.pkl"
model = joblib.load(model_path)

def clean_text(text):
    text = str(text).lower()
    text = re.sub(r"[^a-zA-Z\s]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    words = text.split()
    words = [word for word in words if word not in STOP_WORDS]
    return " ".join(words)

st.set_page_config(
    page_title="Research Abstract Classifier",
```

```
    page_icon="📄"
)

st.title("Researchly Research Abstract Classifier")

st.write(
    "Classify research abstracts as Artificial Intelligence "
    "or Classical Research."
)

abstract = st.text_area(
    "Enter your research abstract:",
    height=250
)

if st.button("Classify Abstract"):
    if not abstract.strip():
        st.warning("Please enter a research abstract.")
    else:
        cleaned_abstract = clean_text(abstract)
        prediction = int(model.predict([cleaned_abstract])[0])

        if prediction == 1:
            st.success("Prediction: ARTIFICIAL INTELLIGENCE (AI)")
        else:
            st.success("Prediction: CLASSICAL RESEARCH")
'''

with open("deployment/classifier.py", "w", encoding="utf-8") as file:
```

```
    file.write(app_code)

print("Streamlit application file created successfully.")
```

Output:

```
Streamlit application file created successfully.
```

```
from importlib.metadata import version

requirements = [
    f"streamlit=={version('streamlit')}",
    f"scikit-learn=={version('scikit-learn')}",
    f"joblib=={version('joblib')}",
    f"nltk=={version('nltk')}"
]

(deployment_folder / "requirements.txt").write_text(
    "\n".join(requirements) + "\n",
    encoding="utf-8"
)

print("Deployment requirements created successfully.")
```

Output:

```
Deployment requirements created successfully.
```

```
#README

readme_content = """
```

```
# Researchly Research Abstract Classifier

This application classifies research abstracts as either
Artificial Intelligence (AI) or Classical Research.

## Files

- classifier.py: Streamlit web application
- best_research_classifier.pkl: Trained machine learning model
- requirements.txt: Required Python packages

## Installation

Install the required packages:

pip install -r requirements.txt

## Run the application

streamlit run classifier.py
"""

(deployment_folder / "README.md").write_text(
    readme_content,
    encoding="utf-8"
)

print("README file created successfully.")
```

Output:

```
README file created successfully.
```

!streamlit run deployment/classifier.py --server.port 8502

Output:



This image shows an abstract from an actual webpage being tested



This image shows an abstract from chat ai being tested.

Question 3

3.1)

```
from pathlib import Path, PurePosixPath

import zipfile

import pandas as pd


zip_path = Path(

  r"C:\Users\Tbose\Downloads\Project 1 Required Resources-
20261102\research_corpus.zip"

)


with zipfile.ZipFile(zip_path, "r") as archive:

  text_files = sorted(

    name for name in archive.namelist()

    if name.lower().endswith(".txt")

  )


  articles = []

  for name in text_files:

    content = archive.read(name).decode("utf-8", errors="replace")

    articles.append({

      "article_id": PurePosixPath(name).stem,

      "filename": name,

      "content": content,

    })


articles_df = pd.DataFrame(articles)
```

```
print("Articles loaded:", len(articles_df))
print("Dataframe shape:", articles_df.shape)
display(articles_df[["article_id", "filename"]])
print("\nFirst article preview:")
print(articles_df.loc[0, "content"][:500])
```

Output:

```
Articles loaded: 5
Dataframe shape: (5, 3)
```

	article_id	filename
0	research_1	research_corpus/research_1.txt
1	research_2	research_corpus/research_2.txt
2	research_3	research_corpus/research_3.txt
3	research_4	research_corpus/research_4.txt
4	research_5	research_corpus/research_5.txt

```
First article preview:
The purposes of this article are to position mixed methods research
(mixed research is a synonym) as the natural complement to traditional qualitative and quantitative research, to present pragmatism
as offering an attractive philosophical partner for mixed methods research, and to provide a framework for designing and conducting
mixed methods research. In doing this, we briefly review the paradigm “wars” and incompatibility thesis, we show some commonalities between quantitativ
e and qualitat
```

3.2)

```
# 3.2
import re
import unicodedata
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

nltk.download("punkt", quiet=True)
nltk.download("stopwords", quiet=True)
nltk.download("punkt_tab", quiet=True)

stop_words = set(stopwords.words("english"))
```

```
def denoise_layout(text):
    text = unicodedata.normalize("NFKC", str(text))
    text = text.replace("\r\n", "\n").replace("\r", "\n")
    text = re.sub(r"[\x00-\x08\x0b\x0c\x0e-\x1f\x7f]", " ", text)
    text = re.sub(r"(?im)^[ \t]*(?:page[ \t]*)?\d+[ \t]*\n?", "", text)
    text = re.sub(r"(?<=\w)-[ \t]*\n[ \t]*(?=[a-z])", "", text)
    text = re.sub(r"(?<!\n)\n(?!\n)", " ", text)
    text = re.sub(r"[ \t]+", " ", text)
    text = re.sub(r" *\n *", "\n", text)
    text = re.sub(r"\n{3,}", "\n\n", text)
    return text.strip()

def prepare_tokens(text):
    words = word_tokenize(text.lower())
    return [
        word for word in words
        if word.isalpha() and word not in stop_words
    ]

articles_df["denoised_content"] = articles_df["content"].apply(denoise_layout)
articles_df["tokens"] = articles_df["denoised_content"].apply(prepare_tokens)
articles_df["preprocessed_content"] = articles_df["tokens"].apply(" ".join)

print("Articles processed:", len(articles_df))
display(
    articles_df.assign(
        token_count=articles_df["tokens"].apply(len)
    )[["article_id", "token_count"]]
```

)

print("\nFirst article token preview:")

print(articles_df.loc[0, "tokens"][:40])

Output:

```
Articles processed: 5
```

	article_id	token_count
0	research_1	3112
1	research_2	1249
2	research_3	2107
3	research_4	3184
4	research_5	2246

```
First article token preview:
['purposes', 'article', 'position', 'mixed', 'methods', 'research', 'mixed', 'research', 'synonym', 'natural', 'complement', 'traditional', 'qualitativ
e', 'quantitative', 'research', 'present', 'pragmatism', 'offering', 'attractive', 'philosophical', 'partner', 'mixed', 'methods', 'research', 'provid
e', 'framework', 'designing', 'conducting', 'mixed', 'methods', 'research', 'briefly', 'review', 'paradigm', 'wars', 'incompatibility', 'thesis', 'sho
w', 'commonalities', 'quantitative']
```

3.3)

```
import numpy as np

import matplotlib.pyplot as plt

from nltk.tokenize import sent_tokenize

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.metrics.pairwise import cosine_similarity


first_article = articles_df.loc[0, "denoised_content"]


sentences = [
    sentence.strip()
    for sentence in sent_tokenize(first_article)
    if len(sentence.split()) > 4
]


vectorizer = TfidfVectorizer(stop_words="english")
```

```
sentence_vectors = vectorizer.fit_transform(sentences)
similarity = cosine_similarity(sentence_vectors)

# Sentence-similarity graph
np.fill_diagonal(similarity, 0)
row_sums = similarity.sum(axis=1, keepdims=True)

transition = np.divide(
  similarity,
  row_sums,
  out=np.zeros_like(similarity),
  where=row_sums != 0,
)



dangling_rows = np.where(row_sums.ravel() == 0)[0]
transition[dangling_rows, :] = 1 / len(sentences)

# TextRank sentence scores
scores = np.full(len(sentences), 1 / len(sentences))
damping = 0.85

for _ in range(100):
  updated_scores = (
    (1 - damping) / len(sentences)
    + damping * transition.T.dot(scores)
  )
```

```
    if np.allclose(scores, updated_scores, atol=1e-6):
        scores = updated_scores
        break

    scores = updated_scores

# Highest-ranked sentences
ranked_indices = np.argsort(scores)[::-1]
selected_indices = []
similarity_threshold = 0.65

for index in ranked_indices:
    if all(
        similarity[index, selected] < similarity_threshold
        for selected in selected_indices
    ):
        selected_indices.append(int(index))

    if len(selected_indices) == 5:
        break


summary_indices = sorted(selected_indices)
first_article_summary = " ".join(
    sentences[index] for index in summary_indices
)

print("TextRank extractive summary:")
```

```
print(first_article_summary)
print(f"\nOriginal words: {len(first_article.split())}")
print(f"Summary words: {len(first_article_summary.split())}")
print(f"Unique sentences selected: {len(summary_indices)}")

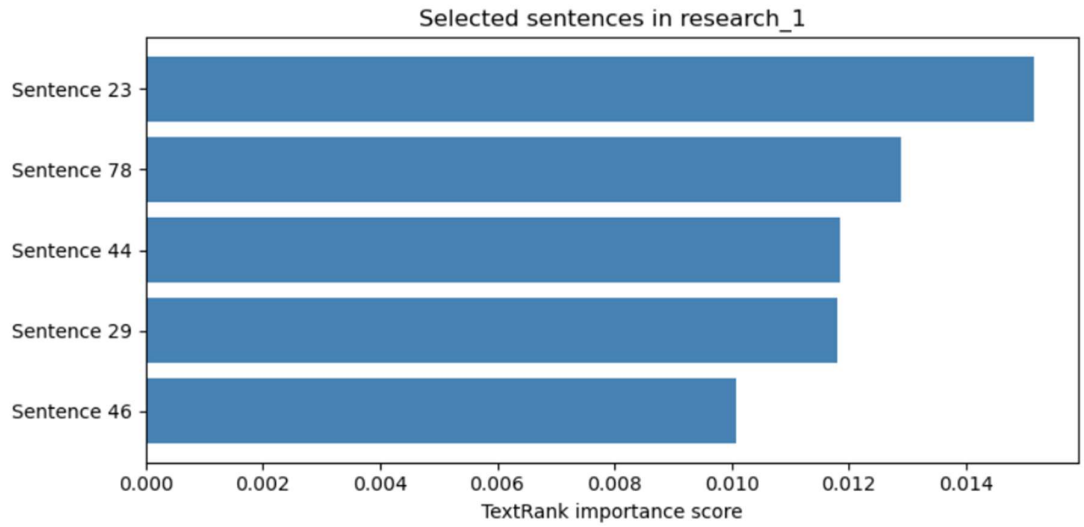
# Visualize the selected sentences' TextRank scores
plot_indices = sorted(selected_indices, key=lambda index: scores[index])

plt.figure(figsize=(8, 4))
plt.barh(
   [f"Sentence {index + 1}" for index in plot_indices],
   [scores[index] for index in plot_indices],
   color="steelblue",
)
plt.xlabel("TextRank importance score")
plt.title("Selected sentences in research_1")
plt.tight_layout()
plt.show()
```

Output:

```
TextRank extractive summary:
Our purpose in writing this article is to present mixed methods research as the third research paradigm in educational research.5 We hope the field wil
l move beyond quantitative versus qualitative research arguments because, as recognized by mixed methods research, both quantitative and qualitative re
search are important and useful. Mixed methods research as the third research paradigm can also help bridge the schism between quantitative and qualita
tive research (Onwuegbuzie & Leech, 2004a). For example, qualitative researchers should be free to use quantitative methods, and quantitative researche
rs should be free to use qualitative methods. We contend that epistemological and paradigmatic ecumenicalism is within reach in the research paradigm o
f mixed methods research. The purposes of this article are to position mixed methods research (mixed research is a synonym) as the natural complement t
o traditional qualitative and quantitative research, to present pragmatism as offering an attractive philosophical partner for mixed methods research,
and to provide a framework for designing and conducting mixed methods research.

Original words: 5600
Summary words: 161
Unique sentences selected: 5
```



3.4)

```
import re
from collections import Counter

import matplotlib.pyplot as plt
from nltk.tokenize import sent_tokenize
from sklearn.feature_extraction.text import TfidfVectorizer

remaining_articles = articles_df.iloc[1:].copy()

def is_complete_sentence(sentence):
    sentence = sentence.strip()
    word_count = len(sentence.split())

    ends_with_punctuation = bool(
        re.search(r'[.!?]["’”\')\]]*$', sentence)
    )
    balanced_parentheses = sentence.count("(") == sentence.count(")")
    balanced_brackets = sentence.count("[") == sentence.count("]")
```

```
    return (
        12 <= word_count <= 45
        and sentence[:1].isupper()
        and ends_with_punctuation
        and balanced_parentheses
        and balanced_brackets
    )

sentence_records = []

for _, row in remaining_articles.iterrows():
    for sentence_number, sentence in enumerate(
        sent_tokenize(row["denoised_content"])
    ):
        sentence = sentence.strip()
        if is_complete_sentence(sentence):
            sentence_records.append({
                "article_id": row["article_id"],
                "sentence_number": sentence_number,
                "sentence": sentence,
            })

if not sentence_records:
    raise ValueError("No complete sentences passed the filters.")

sentences_3_4 = [record["sentence"] for record in sentence_records]
```

```
tfidf_vectorizer = TfidfVectorizer(
   stop_words="english",
   ngram_range=(1, 2),
)
sentence_matrix = tfidf_vectorizer.fit_transform(sentences_3_4)

sentence_scores = []
for row_number in range(sentence_matrix.shape[0]):
   row = sentence_matrix.getrow(row_number)
   sentence_scores.append(row.data.mean() if row.nnz else 0)

# Two high-ranking, non-redundant sentences per article
aggregate_summary = []

for article_id in remaining_articles["article_id"]:
   candidate_indices = [
      i for i, record in enumerate(sentence_records)
      if record["article_id"] == article_id
   ]
   candidate_indices.sort(
      key=lambda i: sentence_scores[i],
      reverse=True,
   )

   selected_for_article = []

   for index in candidate_indices:
      is_redundant = any(
```

```
            sentence_matrix[index]
            .multiply(sentence_matrix[selected])
            .sum() >= 0.55
            for selected in selected_for_article
        )

        if not is_redundant:
            selected_for_article.append(index)

        if len(selected_for_article) == 2:
            break

    for index in sorted(
        selected_for_article,
        key=lambda i: sentence_records[i]["sentence_number"],
    ):
        aggregate_summary.append(sentence_records[index])

print("Aggregate TF-IDF extractive summary:")
for record in aggregate_summary:
    print(f"\n[{record['article_id']}] {record['sentence']}")

generic_terms = {
    "research", "study", "article", "one", "use", "using",
    "also", "however", "result", "results",
}
term_counts = Counter(
    token
```

```
    for tokens in remaining_articles["tokens"]
    for token in tokens
    if token not in generic_terms
)

top_terms = term_counts.most_common(15)

if top_terms:
    terms = [term for term, _ in top_terms][::-1]
    counts = [count for _, count in top_terms][::-1]

    plt.figure(figsize=(9, 6))
    plt.barh(terms, counts, color="darkcyan")
    plt.xlabel("Frequency across articles")
    plt.title("Key terms in research_2 to research_5")
    plt.tight_layout()
    plt.show()

print(f"\nSentences in aggregate summary: {len(aggregate_summary)}")
```

Output:

```
Aggregate TF-IDF extractive summary:

[research_2] These positions of the researcher may impact the research in three major ways.

[research_2] Women to whom I reached out to share with me their experiences were very receptive and cooperative.

[research_3] The focus is no longer only on production, or even on both planning and production.

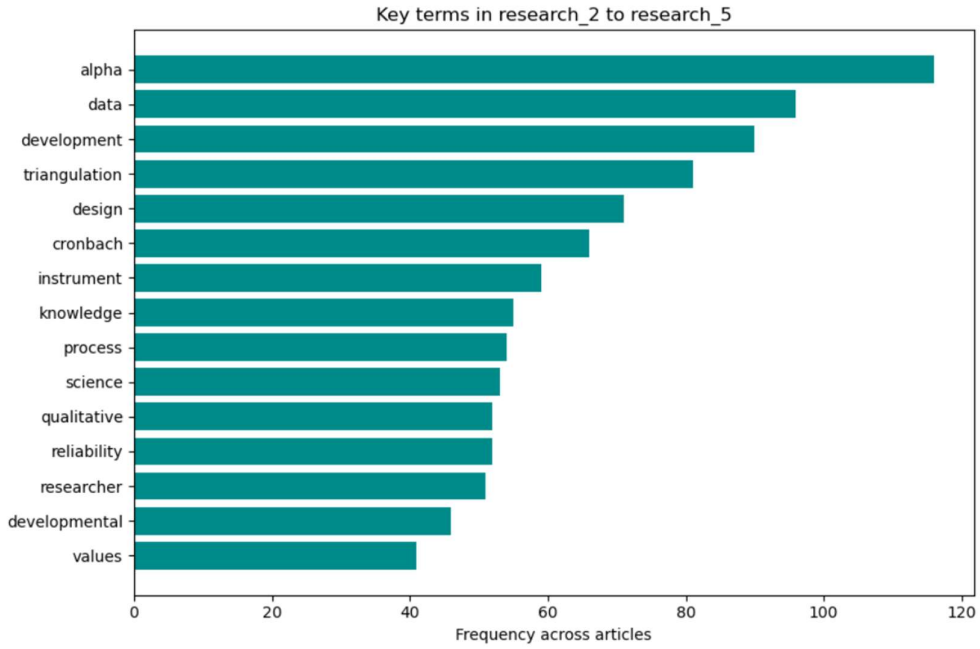
[research_3] In this respect, it is similar to other methodologies such as action research.

[research_4] The most common descriptors were (separately or together) reliability or internal consistency.

[research_4] The authors provided information on how they understood each of these aspects.

[research_5] Rather, the persons with the best skills should be closest to the data.

[research_5] It would be impossible to remove all bias because one is a human being.
```




```
Sentences in aggregate summary: 8
```

Question 4

4.1)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.decomposition import LatentDirichletAllocation

n_topics = 3



count_vectorizer = CountVectorizer(
   max_features=1000,
   max_df=0.95,
   min_df=1,
)

document_term_matrix = count_vectorizer.fit_transform(
   articles_df["preprocessed_content"]
)

lda_model = LatentDirichletAllocation(
   n_components=n_topics,
   random_state=42,
   learning_method="batch",
   max_iter=20,
)
```

```
document_topic_distribution = lda_model.fit_transform(document_term_matrix)
feature_names = count_vectorizer.get_feature_names_out()

# Top words for each topic
for topic_number, topic_weights in enumerate(lda_model.components_, start=1):
    top_word_indices = topic_weights.argsort()[::-1][:10]
    top_words = [feature_names[i] for i in top_word_indices]
    print(f"Topic {topic_number}: {', '.join(top_words)}")

# Each article's topic proportions
topic_columns = [f"Topic {i}" for i in range(1, n_topics + 1)]
topic_distribution_df = pd.DataFrame(
    document_topic_distribution,
    columns=topic_columns,
)
topic_distribution_df.insert(0, "article_id", articles_df["article_id"].values)

print("\nTopic distribution by article:")
display(topic_distribution_df.round(3))

# Topic proportions as a stacked bar chart
x = np.arange(len(articles_df))
bottom = np.zeros(len(articles_df))

plt.figure(figsize=(9, 5))
for topic_index in range(n_topics):
    values = document_topic_distribution[:, topic_index]
```

```
    plt.bar(
        x,
        values,
        bottom=bottom,
        label=f"Topic {topic_index + 1}",
    )
    bottom += values

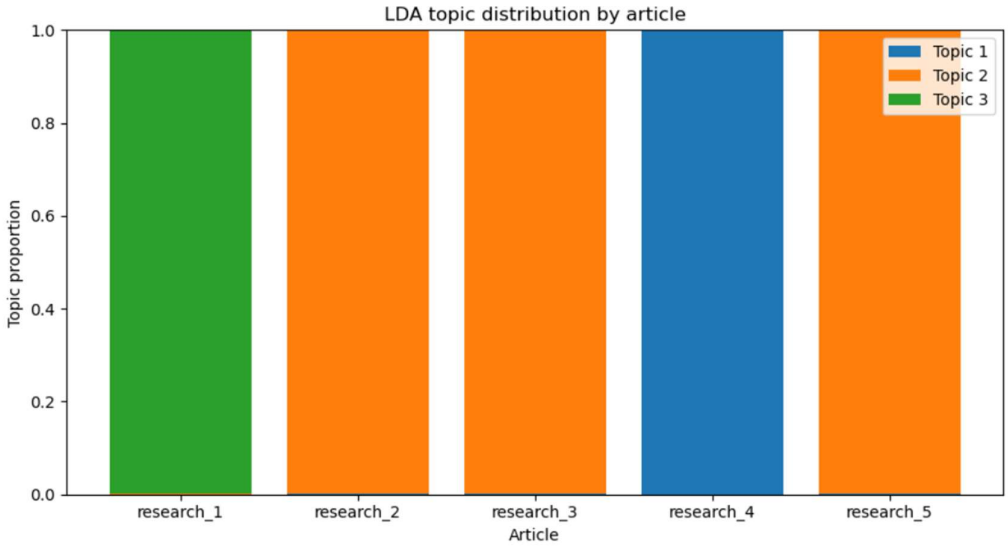
plt.xticks(x, articles_df["article_id"], rotation=0)
plt.ylabel("Topic proportion")
plt.xlabel("Article")
plt.title("LDA topic distribution by article")
plt.ylim(0, 1)
plt.legend()
plt.tight_layout()
plt.show()
```

Output:

```
Topic 1: alpha, cronbach, instrument, science, reliability, education, items, authors, scales, internal
Topic 2: triangulation, development, design, researcher, developmental, instructional, reflexivity, denzin, fusch, researchers
Topic 3: methods, mixed, researchers, quantitative, purists, paradigm, reality, educational, many, paradigms

Topic distribution by article:
```

	article_id	Topic 1	Topic 2	Topic 3
0	research_1	0.000	0.000	1.000
1	research_2	0.001	0.999	0.001
2	research_3	0.000	0.999	0.000
3	research_4	1.000	0.000	0.000
4	research_5	0.000	0.999	0.000



4.2)

```
from pathlib import Path
import pyLDAvis
import pyLDAvis.lda_model
from IPython.display import display

lda_visualization = pyLDAvis.lda_model.prepare(
   lda_model,
   document_term_matrix,
   count_vectorizer,
)

display(pyLDAvis.display(lda_visualization))

output_folder = Path("outputs")
output_folder.mkdir(exist_ok=True)

html_path = output_folder / "lda_intertopic_distance.html"
```

```
pyLDAvis.save_html(lda_visualization, str(html_path))


print(f"Interactive map saved to: {html_path.resolve()}")
```

4.3)

GitHub link

AI Evidence Section

[AI Code 1 – 4 may necessitate evidence submission as specified in your assessment instruction, this section is reserved for evidences where relevant, please delete if not applicable.]

AI Disclosure Appendix

AI tool used	Purpose / how it was used	How I verified or adapted the output
ChatGPT	Natural Language Processing research	Cross-checked with Eduvos notes and textbook material.
ChatGPT	Jupyter notebook Streamlit setup	Cross-checked and internet-searched sources on Streamlit to verify if ChatGPT output was the correct use for the setup
Claude	LDA uses and other examples of text processing, text modelling, and text classification	Cross-checked and internet-searched sources on text processing and text coding sources to verify that Claude's output was valid and correct. Used the output examples as a guide to complete the task in the Assignment

Declaration:

I/we declare that this assessment is my/our own original work, except for source material explicitly acknowledged. I/we confirm that any use of Artificial Intelligence (AI) tools has been applied only as permitted by the assessment instructions and has been fully disclosed where required.

I/we understand that I/we remain responsible for the accuracy, originality, and academic quality of this submission. I/we am/are aware of the institution's policies on academic integrity and the requirements relating to the responsible use of AI in assessments.

I/we acknowledge that failure to comply with these requirements, including undeclared or inappropriate AI use, may result in academic penalties or disciplinary procedures in accordance with institutional policy.

Student Signature(s)	Date
T.Baleni	**01 November 2026**